

Engineering Growth Plan

Practical roadmap for stronger backend, data, cloud and AI engineering skills

Planning date: September 2026

1. The Main Problem to Solve

The goal is not to learn more technologies for the sake of collecting skills. The goal is to become a stronger software engineer who can independently understand, build, debug and reason about production systems.

Core direction: Backend Engineering → Data Engineering → Cloud/Distributed Systems, with AI/agent systems as a complementary capability.

2. What to Prioritize

Priorit y	Focus	Why
1	Python + Backend Engineering	Restore confidence in fundamentals and make implementation genuinely yours.
2	Data Engineering	Build toward Big Data, Spark, ETL and data-platform problems.
3	PostgreSQL + Data Systems	Move from SQL exercises to real database design, indexing and performance.
4	System Design	Connect APIs, databases, queues, storage, caching and scaling into complete systems.
5	Cloud + DevOps	Turn existing AWS/Docker/K8s knowledge into practical deployment ability.
6	AI / Agentic AI	Keep learning it, but focus on understanding the engineering underneath the abstractions.

3. Phase 1 — Rebuild Implementation Confidence

Duration: ~6–8 weeks

- **Python:** write small programs without an LLM first — files, pathlib, JSON, CSV, exceptions, iterators, generators, decorators, typing and Pydantic.
- **Backend:** build APIs using FastAPI with proper validation, error handling, dependency management and testing.
- **SQLAlchemy:** understand what the Python abstraction is actually doing underneath: SQL, transactions, relationships, joins and connection management.
- **GitHub Actions:** learn the broader model — triggers, jobs, runners, matrices, artifacts, environments, secrets, reusable workflows and deployment pipelines.
- **Rule:** documentation lookup is completely fine. The objective is to know the concept and be able to implement a reasonable solution, not memorize syntax.

4. Phase 2 — Start the Data Engineering Track

Duration: ~3–4 months

- **SQL depth:** joins, CTEs, window functions, query plans, indexes, transactions, isolation and performance.
- **Python data processing:** build ETL jobs that ingest, validate, transform and persist data.
- **PySpark:** DataFrames, transformations/actions, partitions, shuffles, joins, caching, serialization and performance.
- **Data pipelines:** understand batch vs streaming, orchestration, retries, idempotency, data quality and observability.

- **Project:** create one realistic pipeline rather than several tutorial projects — ingest data → validate → transform → store → expose/query → monitor.

5. Phase 3 — Turn System Design Into Engineering

- Design systems using components you have actually implemented: API, PostgreSQL, cache, object storage, queue, worker and monitoring.
- For every design, ask: What happens when traffic increases? A dependency fails? A message is duplicated? A database becomes slow? A request is retried?
- Learn practical concepts: caching, rate limiting, asynchronous processing, queues, Kafka fundamentals, consistency, partitioning, replication and observability.
- Write short architecture notes for your own projects. Focus on trade-offs rather than polished diagrams.

6. Phase 4 — Cloud & Infrastructure Depth

- Build on the AWS Solutions Architect knowledge by deploying your own application rather than collecting additional certifications immediately.
- Use Docker → Kubernetes → Helm → AWS as one connected learning path.
- Then deepen the gaps: Argo CD, Prometheus, Grafana, logging, IAM, networking, load balancing and managed database services.
- Understand what the platform is doing rather than relying on generated configuration.

7. AI / Agentic AI — Keep It, But Change the Learning Style

Do not abandon AI/agentic AI. Instead, stop treating an AI-generated project as evidence that you understand the system. Build smaller components yourself and use AI as an assistant.

- Understand RAG end-to-end: ingestion, chunking, embeddings, vector search, retrieval, reranking and generation.
- Understand MCP and agent workflows conceptually: tools, context, orchestration, state, failure handling and permissions.
- When using Claude Code or another coding assistant, first predict the implementation, then inspect the generated code line by line, then modify it yourself.
- Maintain one small AI project alongside the main backend/data-engineering project rather than making AI the entire roadmap.

8. The Most Important Change: How You Learn

- **Before AI:** spend 20–30 minutes attempting the implementation yourself.
- **During AI assistance:** ask for explanation, alternatives and reasoning — not simply finished code.
- **Afterwards:** close the generated answer and recreate the important part yourself.
- **For unfamiliar code:** trace the execution path manually and identify inputs, outputs, dependencies and failure cases.
- **For senior engineers:** ask why an approach was selected, what trade-off was accepted and what they would change with hindsight.

9. Weekly Operating Plan

Activity	Target
DSA	3–4 focused sessions/week; finish Trees, Graphs and DP systematically.
Backend / Python	3 sessions/week; implement without AI first.
Data Engineering	4–5 sessions/week as the main long-term track.
System Design	2 sessions/week; design systems using technologies already learned.
AI / MCP	1–2 sessions/week; maintain continuity without letting it dominate.
Project	Continuous; one serious project, developed incrementally.

10. What “Good Progress” Looks Like

- You can write ordinary Python and backend code without needing an LLM for every implementation detail.
- You can explain what SQLAlchemy is doing underneath the Python code.
- You can design and implement a real ETL pipeline and explain its failure modes.
- You can use Spark and explain why a job is slow, where shuffles occur and how partitioning affects it.
- You can design a backend system and defend your architectural choices.
- You can deploy and observe your own service on cloud/container infrastructure.
- You can read AI-generated code critically instead of feeling that the project belongs to the AI.

11. What NOT to Do

- Do not start five new technologies simultaneously.
- Do not collect certifications as a substitute for implementation.
- Do not build many tiny AI demos with mostly generated code.
- Do not spend months only solving interview questions while your engineering confidence remains weak.
- Do not try to memorize every command or API. Learn the model, then use documentation for syntax.
- Do not measure progress by how many courses you completed. Measure it by what you can build and explain without assistance.

12. End Goal

Near term: become a dependable, technically strong software engineer with solid backend, data, cloud and system-design fundamentals.

Longer term: develop the breadth and depth required to reason about larger systems and eventually move toward architecture-level responsibilities.

The key is not to become an expert in everything. It is to build one strong engineering foundation and then layer data, cloud, distributed systems and AI capabilities on top of it.